Stat534 HW4

Packages

```
library(data.table)
library(MASS)
library(snow)
```

Data

```
df <- fread("534binarydata.txt")
```

Problem 1

Part A: Solution

```
# Problem 1 ------------------------------------------------------------------------------------

#' Goal: function computing the Lapalace approximation of the marginal likelihood
#' Setup: univariate logistic regression model


getLaplaceApprox <- function(response, explanatory, data, betaMode) {

    # data
    xy <- get_xy(response, explanatory, data)
    y <- xy$y
    x <- xy$x

    # hessian
    hess_mode <- get_post_hessian(x=x, y=y, betas=betaMode)
    neg_hess <- -hess_mode

    # log deteriminant
    log_det <- as.numeric(
        determinant(neg_hess, logarithm = TRUE)$modulus
    )

    log_marginal_lik <- log(2 * pi) + post_loglik(x=x, y=y, betas=betaMode) - 0.5 * log_det

    return(log_marginal_lik)
}

# Helpers --------------------------------------------------------------------------------------
```

```r
# 1. get proper data (X, y)
get_xy <- function(response, explanatory, data) {
    m <- as.matrix(data)
    return(list(
        x=m[, explanatory],
        y=m[, response]
    ))
}


# 2. log-likelihood functions

log1pexp <- function(x) {
    # if (x > 0) {
    #     return(x + log1p(exp(-x)))
    # }
    # return(log1p(exp(x)))
    #NOTE: need to vectorize func, above is not
    ifelse(x > 0, x + log1p(exp(-x)), log1p(exp(x)))
}


post_loglik <- function(x, y, betas) {

    beta0 <- betas[1]
    beta1 <- betas[2]

    temp <- beta0 + beta1 * x

    loglik <- sum(y * temp - log1pexp(temp))
    log_prior <- -log(2 * pi) - 0.5 * sum(betas**2)
    log_post <- log_prior + loglik

    return(log_post)
}

get_post_gradient <- function(x, y, betas) {

    beta0 <- betas[1]
    beta1 <- betas[2]

    temp <- beta0 + beta1 * x
    pi_ <- plogis(temp)

    grad0 <- sum(y - pi_) - beta0
    grad1 <- sum((y - pi_) * x) - beta1

    return(c(grad0, grad1))
}

get_post_hessian <- function(x, y, betas) {

    beta0 <- betas[1]
```

```
    beta1 <- betas[2]

    temp <- beta0 + beta1 * x
    pi_ <- plogis(temp)
    w <- pi_ * (1 - pi_)

    h00 <- -sum(w) - 1
    h01 <- -sum(w * x)
    h11 <- -sum(w * x^2) - 1

    hess <- matrix(c(h00, h01,
             h01, h11), nrow = 2, byrow = TRUE)

    return(hess)
}
```

```
getLaplaceApprox(ncol(df), 1, data = df, betaMode = c(0, 0))
```

```
## [1] -106.2201
```

Part B: AI Solution

Prompt

I need you to design a function in R that computes the Laplace approximation of the marginal likelihood associated with a univariate logistic regression. The function should be designed as follow

“ “ “getLaplaceApprox <- function(response, explanatory, data, betaMode)” “ ”

where the inputs are: response: integer - column index corresponding to the response variable, explanatory: integer- column index corresponding to an explanatory variable (note we are in an univariate model), data: data frame or matrix - contains all the variables, betaMode: vector - mode of the posterior distribution from Newton-Raphson. Your function should return a number. REMEBER for prior π, please use `pi_` as the variable name to avoid using built-in pi constant.

Code

```
# Problem 1 ------------------------------------------------------------------------------

AI_getLaplaceApprox <- function(response, explanatory, data, betaMode) {
    # Extract variables
    y <- data[, response]
    x <- data[, explanatory]
    n <- length(y)

    b0 <- betaMode[1]
    b1 <- betaMode[2]

    # Calculate pi_i (inverse logit) [cite: 15, 21]
    # Using pi_ to avoid conflict with R's built-in pi [user request]
    linear_pred <- b0 + b1 * x
    pi_ <- exp(linear_pred) / (1 + exp(linear_pred))

    # Calculate log-likelihood l(beta|D) [cite: 25]
    # We use a small epsilon to avoid log(0)
```

```
    eps <- 1e-16
    log_lik <- sum(y * log(pi_ + eps) + (1 - y) * log(1 - pi_ + eps))

    # Calculate l*(beta)
    # l*(beta) = -log(2*pi) - 0.5*(beta0^2 + beta1^2) + l(beta|D)
    l_star <- -log(2 * base::pi) - 0.5 * (b0^2 + b1^2) + log_lik

    # Calculate the Hessian matrix D^2 l*(beta) [cite: 38, 39]
    # From log-likelihood derivatives[cite: 26], the second derivatives are:
    # d2l/db0^2 = -sum(pi * (1-pi))
    # d2l/db0db1 = -sum(x * pi * (1-pi))
    # d2l/db1^2 = -sum(x^2 * pi * (1-pi))
    # The prior adds -1 to the diagonal elements of the Hessian

    weight <- pi_ * (1 - pi_)
    h00 <- -sum(weight) - 1
    h01 <- -sum(x * weight)
    h11 <- -sum(x^2 * weight) - 1

    hessian <- matrix(c(h00, h01, h01, h11), nrow = 2)

    # Calculate log marginal likelihood [cite: 52, 55]
    # log(2*pi) + l_star - 0.5 * log(det(-Hessian))
    det_neg_hessian <- det(-hessian)
    log_marg_lik <- log(2 * base::pi) + l_star - 0.5 * log(det_neg_hessian)

    return(log_marg_lik)
}
```

Part C: Comparison

```
test_idx <- sample(1:(ncol(df) - 1), size = 1)

getLaplaceApprox(ncol(df), test_idx, data = df, betaMode = c(0, 0))
```

```
## [1] -106.2201
```

```
AI_getLaplaceApprox(ncol(df), test_idx, data = as.matrix(df), betaMode = c(0, 0))
```

```
## [1] -106.2201
```

- The AI solution and my solution implement the same Laplace approximation logic (following same instructions). Both compute the posterior mode, evaluate the log posterior at that mode, compute the Hessian of the log posterior, and use the determinant of the negative Hessian in the approximation.
- The AI code is correct for this problem. I randomly checked the predictors in `534binarydata.txt`. The Laplace log marginal likelihoods are equal.
- My solution is more modular with helper functions such as `get_xy()`, `post_loglik()`. The AI solution repeats some calc. directly inside the function, which makes the function self-contained but have duplicated code.
- Regarding the stability. I use `plogis()` and the stable expression `log1pexp(eta)` for the logistic log likelihood. The AI code computes `exp(eta) / (1 + exp(eta))` and then adds a small epsilon inside `log()`, which works on this data set but is less reliable for very large positive or negative linear predictors.

- Both implementations use the same data structures: numeric vectors for `x`, `y`, and `betaMode`, and a `2 x 2` numeric matrix for the Hessian. The algorithmic complexity is essentially the same.

Problem 2

Part A: Solution

```
# Problem 2 ------------------------------------------------------------------------------------

#' Goal: estimate coefficients betas with Metropolis-Hasting Sampling

getPosteriorMeans <- function(response, explanatory, data, betaMode, niter) {

    # data
    xy <- get_xy(response, explanatory, data)
    y <- xy$y
    x <- xy$x

    # setups
    hess_mode <- get_post_hessian(x, y, betas=betaMode)
    proposal_cov <- -solve(hess_mode)

    beta_chain <- matrix(NA_real_, nrow = niter, ncol = length(betaMode))
    colnames(beta_chain) <- paste0("beta", (1:length(betaMode))-1)

    # initialization
    beta_curr <- betaMode
    logpost_curr <- post_loglik(x=x, y=y, betas=beta_curr)

    # main sampling
    for (iter in 1:niter) {
        # sample proposal dist.
        beta_prop <- as.numeric(MASS::mvrnorm(
            n = 1,
            mu = beta_curr,
            Sigma = proposal_cov
        ))

        logpost_prop <- post_loglik(x, y, betas=beta_prop)
        log_accept_prob <- (logpost_prop - logpost_curr)

        if (log(runif(1)) <= log_accept_prob) {
            # case: accept w/ prob.
            beta_curr <- beta_prop
            logpost_curr <- logpost_prop
        }
        # case: reject
        beta_chain[iter, ] <- beta_curr
    }

    return(beta_chain)
}
```

The (β_0, β_1) from Metropolis-Hasting is $(-0.7570194, 1.0153382)$.

```
beta_chain <- getPosteriorMeans(
    response = ncol(df),
    explanatory = 1,
```

```
    data = df,
    betaMode = c(0, 0),
    niter = 10000
)
colMeans(beta_chain)
```

```
##      beta0      beta1
## -0.7576401  0.9994480
```

Part B: AI Solution

Prompt

Write a function in R that estimates the coefs beta0 and beta1 of the univariate logistic regression (1) as follows: 1. Use the Metropolis-Hastings algorithm to simulate 10000 samples 2. The esetimates for beta0 and beta1 are given by the sample means

you will write a function getPosteriorMeans as follows: getPosteriorMeans <- function(response,explanatory,data, betaMode, niter) where the inputs are: response: integer - column index corresponding to the response variable, explanatory: integer- column index corresponding to an explanatory variable (note we are in an univariate model), data: data frame or matrix - contains all the variables, betaMode: vector - mode of the posterior distribution from Newton-Raphson, niter: number of iterations for Metropolis-Hastings. Your function should return a matrix or a vector of Beta values.

Code

```
# Problem 2 ------------------------------------------------------------------------------------

AI_getPosteriorMeans <- function(response, explanatory, data, betaMode, niter) {

    # Extract data [cite: 7]
    y <- data[, response]
    x <- data[, explanatory]

    # 1. Setup the Proposal Covariance Matrix [cite: 65]
    # We need the Hessian at the mode to define the proposal distribution
    b0_mode <- betaMode[1]
    b1_mode <- betaMode[2]

    # Calculate pi_i at the mode [cite: 15, 21]
    lin_pred_mode <- b0_mode + b1_mode * x
    pi_mode <- exp(lin_pred_mode) / (1 + exp(lin_pred_mode))

    # Compute Hessian D^2 l*(beta) at the mode [cite: 39]
    weight_mode <- pi_mode * (1 - pi_mode)
    h00 <- -sum(weight_mode) - 1
    h01 <- -sum(x * weight_mode)
    h11 <- -sum(x^2 * weight_mode) - 1
    hessian_mode <- matrix(c(h00, h01, h01, h11), nrow = 2)

    # Proposal covariance is the negative inverse of the Hessian [cite: 65]
    prop_sigma <- solve(-hessian_mode)

    # 2. Define the l*(beta) function for log-posterior density [cite: 32]
    get_l_star <- function(beta_vec) {
```

```
        b0 <- beta_vec[1]
        b1 <- beta_vec[2]

        lp <- b0 + b1 * x
        # Using pi_ to avoid built-in pi constant [user instruction]
        pi_ <- exp(lp) / (1 + exp(lp))

        # Log-likelihood [cite: 25]
        # Add small epsilon to avoid log(0)
        log_lik <- sum(y * log(pi_ + 1e-16) + (1 - y) * log(1 - pi_ + 1e-16))

        # Return l*(beta) [cite: 32]
        return(-log(2 * base::pi) - 0.5 * (b0^2 + b1^2) + log_lik)
    }

    # 3. Metropolis-Hastings Algorithm [cite: 57, 75]
    # Initialize storage and starting values [cite: 58, 94]
    samples <- matrix(0, nrow = niter, ncol = 2)
    samples[1, ] <- betaMode
    current_l_star <- get_l_star(betaMode)

    for (k in 2:niter) {
        # Generate candidate from bivariate normal [cite: 62, 63]
        candidate <- MASS::mvrnorm(n = 1, mu = samples[k - 1, ], Sigma = prop_sigma)
        candidate_l_star <- get_l_star(candidate)

        # Calculate log acceptance probability [cite: 70, 82]
        log_alpha <- candidate_l_star - current_l_star

        # Acceptance step [cite: 78, 81, 84, 86]
        if (log(runif(1)) <= log_alpha) {
            samples[k, ] <- candidate
            current_l_star <- candidate_l_star
        } else {
            samples[k, ] <- samples[k - 1, ]
        }
    }

    # 4. Calculate posterior means
    beta0_mean <- mean(samples[, 1])
    beta1_mean <- mean(samples[, 2])

    return(c(beta0 = beta0_mean, beta1 = beta1_mean))
}
```

Part C: Comparison

```
colMeans(beta_chain)
```

```
##      beta0      beta1
## -0.7576401  0.9994480
```

```
AI_beta_chain <- AI_getPosteriorMeans(
    response = ncol(df),
```

```
    explanatory = 1,
    data = as.matrix(df),
    betaMode = c(0, 0),
    niter = 10000
)
AI_beta_chain
```

```
##      beta0      beta1
## -0.7573178  1.0093119
```

- The AI solution uses the same Metropolis-Hastings algo. as my solution. Both have proposal distribution, proposal covariance, and an acceptance probability based on the log posterior difference.
- The AI code is the same as my solution with negliable differences. (<0.1)
- The main output difference is the return value. My `getPosteriorMeans()` returns the full Markov chain, and the posterior means are computed later. The AI function returns only a length-2 vector of posterior means. The AI's output does not specifically follow the requirements.
- Both solutions use a numeric matrix to store sampled beta values, a numeric vector for each proposed candidate, and a `2 x 2` matrix for the proposal covariance. The algorithms are the same Metropolis-Hastings algorithm, except that the AI chain stores `betaMode` as the first row and then runs from iteration 2, while my code records a state after each proposal step.

Problem 3

Part A: Solution

I redesgined the main function output, since the original one is bad for later use and check.

```
# Problem 3 ------------------------------------------------------------------------------------

#' Goal: bayes logistic model with 1 variable, 1 outcome

bayesLogistic <- function(apredictor, response, data, NumberOfIterations) {

    # get beta mode
    betaMode <- get_betas_mode_NR(
        response = response,
        explanatory = apredictor,
        data = data
    )

    # calc. loglik
    log_marginal_lik <- getLaplaceApprox(
        response = response,
        explanatory = apredictor,
        data = data,
        betaMode = betaMode
    )

    # chain
    beta_chain <- getPosteriorMeans(
        response = response,
        explanatory = apredictor,
        data = data,
        betaMode = betaMode,
        niter = NumberOfIterations
    )
    beta_bayes <- colMeans(beta_chain)

    # mle
    dat_mat <- as.matrix(data)
    y <- dat_mat[, response]
    x <- dat_mat[, apredictor]
    mle_fit <- glm(y ~ x, family = binomial())
    mle_coef <- coef(mle_fit)

    # results
    res <- list(
        apredictor=apredictor,
        logmarglik=log_marginal_lik,
        beta0bayes=unname(beta_bayes[1]),
        beta1bayes=unname(beta_bayes[2]),
        beta0mle=unname(mle_coef[1]),
        beta1mle=unname(mle_coef[2])
    )

    return(res)
```

```
}


# Algo: Newton-Ralphson ------------------------------------------------------------------------

get_betas_mode_NR <- function(
        response, explanatory, data,
        tol=1e-4, max_iter=100, verbose=FALSE
    ) {

    # get data
    xy <- get_xy(response, explanatory, data)
    x <- xy$x
    y <- xy$y

    # initialization
    betas <- c(0, 0)

    # main
    for (iter in 1:max_iter) {
        grad <- get_post_gradient(x=x, y=y, betas=betas)
        hess <- get_post_hessian(x=x, y=y, betas=betas)

        betas_new <- betas - solve(hess, grad) # NOTE: better than solve(hess) %*% grad

        # early stop condition
        if (max(abs(betas_new - betas)) < tol) {
            if (verbose) {cat(sprintf("Converge at iter=%d", iter))}
            return(betas_new)
        }

        # update parameters
        betas <- betas_new
    }
    cat("NR Algo did not converge")
    return(betas)
}


# Main Parallel Function

main <- function(datafile, NumberOfIterations, clusterSize) {

    # load data
    data <- read.table(datafile, header = FALSE)

    # extract variables
    response <- ncol(data)
    lastPredictor <- ncol(data) - 1

    # cluster setups
    cluster <- makeCluster(clusterSize, type="SOCK")
```

```
    clusterEvalQ(
        cluster,
        {library(MASS)}
    ) # run expression on each cluster

    clusterExport(cluster, c(
        "get_xy",
        "log1pexp",
        "post_loglik",
        "get_post_gradient",
        "get_post_hessian",
        "get_betas_mode_NR",
        "getLaplaceApprox",
        "getPosteriorMeans",
        "bayesLogistic"
    )) # export helpers to each cluster


    # Main estimation
    results <- clusterApply(
        cluster,
        1:lastPredictor,
        bayesLogistic,
        response,
        data,
        NumberOfIterations
    )

    #destroy the cluster
    stopCluster(cluster);

    return(results)
}
```

```
res <- main("534binarydata.txt", 10000, 10)
res_df <- rbindlist(res)
res_df |> head()
```

```
##    apredictor logmarglik beta0bayes beta1bayes   beta0mle   beta1mle
##         <int>      <num>      <num>      <num>      <num>      <num>
## 1:          1  -84.17025 -0.7578759  1.0279014 -0.7845448  1.0422840
## 2:          2  -91.13804 -0.7889423 -0.6635927 -0.8117805 -0.6834140
## 3:          3  -89.27079 -0.8017210  0.7613599 -0.8174069  0.7676812
## 4:          4  -94.74949 -0.7436699  0.3668977 -0.7581619  0.3837138
## 5:          5  -92.78677 -0.7603837  0.5410731 -0.7836879  0.5479422
## 6:          6  -94.44806 -0.7469039  0.3997014 -0.7575917  0.4039104
```

Part B: AI Solution

Prompt

write a function bayesLogistic as follows: bayesLogistic <- function(apredictor, response, data, NumberOfIterations) where the inputs are: apredictor: integer - column index corresponding to an explanatory variable (note we are in an univariate model), response: integer - column index corresponding to the response variable, data: data frame or matrix - contains all the variables, NumberOfIterations:number of iterations for

Metropolis-Hastings. Your function should return a list with the following elements as stated in the sample main function: apredictor:integer - index of predictor, logmarglik:real number - log marginal likelihood, beta0bayes:real number - beta0 estimate from posterior, beta1bayes:real number - beta1 estimate from posterior, beta0mle (optional):real number - beta0 MLE estimate, beta1mle (optional):real number - beta1 MLE estimate

Also, compose a main function following this parallel framework:

Code

```
# baye logistic
AI_bayesLogistic <- function(apredictor, response, data, NumberOfIterations) {
    # 1. Find the mode via Newton-Raphson
    beta_mode <- getMode(response, apredictor, data)

    # 2. Compute Laplace Approximation (Log Marginal Likelihood) [cite: 89]
    log_ml <- AI_getLaplaceApprox(response, apredictor, data, beta_mode)

    # 3. Compute Posterior Means via Metropolis-Hastings [cite: 90, 91]
    post_means <- AI_getPosteriorMeans(response, apredictor, data, beta_mode, NumberOfIterations)

    # 4. Optional: Calculate MLEs using standard logistic regression
    fit_mle <- glm(data[, response] ~ data[, apredictor], family = "binomial")
    mles <- coef(fit_mle)

    # Return the results as a list [cite: 135, 136]
    return(list(
        apredictor = apredictor,
        logmarglik = log_ml,
        beta0bayes = post_means[1],
        beta1bayes = post_means[2],
        beta0mle = mles[1],
        beta1mle = mles[2]
    ))
}


# main
library(snow) # Required for parallel computing [cite: 140]

AI_main <- function(datafile, NumberOfIterations, clusterSize) {
    data <- read.table(datafile, header = FALSE)

    response_idx <- ncol(data)
    lastPredictor <- ncol(data) - 1

    cluster <- makeCluster(clusterSize, type = "SOCK")

    clusterExport(
        cluster,
        c("getMode", "AI_getLaplaceApprox", "AI_getPosteriorMeans", "AI_bayesLogistic")
    )

    results <- clusterApply(
```

```
        cluster,
        1:lastPredictor,
        AI_bayesLogistic,
        response_idx,
        data,
        NumberOfIterations
    )

    for (i in 1:lastPredictor) {
        cat(
            "Regression of Y on explanatory variable", results[[i]]$apredictor,
            "has log marginal likelihood", results[[i]]$logmarglik,
            "with beta0 =", results[[i]]$beta0bayes, "(", results[[i]]$beta0mle, ")",
            "and beta1 =", results[[i]]$beta1bayes, "(", results[[i]]$beta1mle, ")",
            "\n"
        )
    }

    stopCluster(cluster)
    return(results)
}

# Helpers
# Find the mode of the posterior using Newton-Raphson
getMode <- function(response, explanatory, data) {
    y <- data[, response]
    x <- data[, explanatory]
    beta <- c(0, 0) # Initial values (0,0) [cite: 44]
    eps <- 0.0001
    diff <- 1

    while(diff > eps) {
        beta_old <- beta
        lp <- beta[1] + beta[2] * x
        pi_ <- exp(lp) / (1 + exp(lp)) # Inverse logit [cite: 21]

        # Gradient of l* [cite: 26, 37]
        grad <- c(sum(y - pi_) - beta[1], sum(y * x - pi_ * x) - beta[2])

        # Hessian of l* [cite: 38, 39]
        weight <- pi_ * (1 - pi_)
        h00 <- -sum(weight) - 1
        h01 <- -sum(x * weight)
        h11 <- -sum(x^2 * weight) - 1
        hessian <- matrix(c(h00, h01, h01, h11), nrow = 2)

        # Update step [cite: 46]
        beta <- beta - solve(hessian) %*% grad
        diff <- max(abs(beta - beta_old)) # Convergence check [cite: 47]
    }
    return(as.vector(beta))
}
```

Part C: Comparison

```
AI_res <- AI_main("534binarydata.txt", 10000, 10)
```

```
## Regression of Y on explanatory variable 1 has log marginal likelihood -84.17025 with beta0 = -0.7477
## Regression of Y on explanatory variable 2 has log marginal likelihood -91.13804 with beta0 = -0.7825
## Regression of Y on explanatory variable 3 has log marginal likelihood -89.27079 with beta0 = -0.7941
## Regression of Y on explanatory variable 4 has log marginal likelihood -94.74949 with beta0 = -0.7367
## Regression of Y on explanatory variable 5 has log marginal likelihood -92.78677 with beta0 = -0.7688
## Regression of Y on explanatory variable 6 has log marginal likelihood -94.44806 with beta0 = -0.7397
## Regression of Y on explanatory variable 7 has log marginal likelihood -95.40641 with beta0 = -0.7330
## Regression of Y on explanatory variable 8 has log marginal likelihood -88.02388 with beta0 = -0.7799
## Regression of Y on explanatory variable 9 has log marginal likelihood -95.46509 with beta0 = -0.6983
## Regression of Y on explanatory variable 10 has log marginal likelihood -94.84316 with beta0 = -0.733
## Regression of Y on explanatory variable 11 has log marginal likelihood -91.98299 with beta0 = -0.781
## Regression of Y on explanatory variable 12 has log marginal likelihood -96.1455 with beta0 = -0.7296
## Regression of Y on explanatory variable 13 has log marginal likelihood -92.13999 with beta0 = -0.770
## Regression of Y on explanatory variable 14 has log marginal likelihood -95.39082 with beta0 = -0.732
## Regression of Y on explanatory variable 15 has log marginal likelihood -91.25791 with beta0 = -0.803
## Regression of Y on explanatory variable 16 has log marginal likelihood -93.49182 with beta0 = -0.761
## Regression of Y on explanatory variable 17 has log marginal likelihood -91.31417 with beta0 = -0.785
## Regression of Y on explanatory variable 18 has log marginal likelihood -89.08946 with beta0 = -0.794
## Regression of Y on explanatory variable 19 has log marginal likelihood -90.42463 with beta0 = -0.752
## Regression of Y on explanatory variable 20 has log marginal likelihood -94.86778 with beta0 = -0.738
## Regression of Y on explanatory variable 21 has log marginal likelihood -85.74706 with beta0 = -0.832
## Regression of Y on explanatory variable 22 has log marginal likelihood -84.03714 with beta0 = -0.847
## Regression of Y on explanatory variable 23 has log marginal likelihood -79.48414 with beta0 = -0.895
## Regression of Y on explanatory variable 24 has log marginal likelihood -94.15746 with beta0 = -0.749
## Regression of Y on explanatory variable 25 has log marginal likelihood -96.58153 with beta0 = -0.717
## Regression of Y on explanatory variable 26 has log marginal likelihood -91.73426 with beta0 = -0.744
## Regression of Y on explanatory variable 27 has log marginal likelihood -90.40866 with beta0 = -0.783
## Regression of Y on explanatory variable 28 has log marginal likelihood -95.21228 with beta0 = -0.740
## Regression of Y on explanatory variable 29 has log marginal likelihood -91.99002 with beta0 = -0.765
## Regression of Y on explanatory variable 30 has log marginal likelihood -90.80862 with beta0 = -0.782
## Regression of Y on explanatory variable 31 has log marginal likelihood -90.88962 with beta0 = -0.784
## Regression of Y on explanatory variable 32 has log marginal likelihood -90.48714 with beta0 = -0.810
## Regression of Y on explanatory variable 33 has log marginal likelihood -95.73493 with beta0 = -0.730
## Regression of Y on explanatory variable 34 has log marginal likelihood -89.09465 with beta0 = -0.823
## Regression of Y on explanatory variable 35 has log marginal likelihood -96.68771 with beta0 = -0.723
## Regression of Y on explanatory variable 36 has log marginal likelihood -96.73543 with beta0 = -0.728
## Regression of Y on explanatory variable 37 has log marginal likelihood -83.42107 with beta0 = -0.845
## Regression of Y on explanatory variable 38 has log marginal likelihood -93.49103 with beta0 = -0.772
## Regression of Y on explanatory variable 39 has log marginal likelihood -87.4932 with beta0 = -0.8174
## Regression of Y on explanatory variable 40 has log marginal likelihood -91.01029 with beta0 = -0.775
## Regression of Y on explanatory variable 41 has log marginal likelihood -91.2524 with beta0 = -0.7539
## Regression of Y on explanatory variable 42 has log marginal likelihood -85.78488 with beta0 = -0.843
## Regression of Y on explanatory variable 43 has log marginal likelihood -94.16321 with beta0 = -0.746
## Regression of Y on explanatory variable 44 has log marginal likelihood -95.9192 with beta0 = -0.7315
## Regression of Y on explanatory variable 45 has log marginal likelihood -95.12644 with beta0 = -0.746
## Regression of Y on explanatory variable 46 has log marginal likelihood -86.60919 with beta0 = -0.828
## Regression of Y on explanatory variable 47 has log marginal likelihood -91.62257 with beta0 = -0.771
## Regression of Y on explanatory variable 48 has log marginal likelihood -91.46699 with beta0 = -0.776
## Regression of Y on explanatory variable 49 has log marginal likelihood -90.41233 with beta0 = -0.776
## Regression of Y on explanatory variable 50 has log marginal likelihood -92.4211 with beta0 = -0.7539
```

```
## Regression of Y on explanatory variable 51 has log marginal likelihood -95.32329 with beta0 = -0.743
## Regression of Y on explanatory variable 52 has log marginal likelihood -93.79355 with beta0 = -0.752
## Regression of Y on explanatory variable 53 has log marginal likelihood -93.90333 with beta0 = -0.757
## Regression of Y on explanatory variable 54 has log marginal likelihood -91.25855 with beta0 = -0.782
## Regression of Y on explanatory variable 55 has log marginal likelihood -90.68376 with beta0 = -0.784
## Regression of Y on explanatory variable 56 has log marginal likelihood -90.96067 with beta0 = -0.765
## Regression of Y on explanatory variable 57 has log marginal likelihood -95.2976 with beta0 = -0.7353
## Regression of Y on explanatory variable 58 has log marginal likelihood -92.44812 with beta0 = -0.758
## Regression of Y on explanatory variable 59 has log marginal likelihood -97.02482 with beta0 = -0.717
## Regression of Y on explanatory variable 60 has log marginal likelihood -93.98176 with beta0 = -0.737
```

```
AI_res_df <- rbindlist(AI_res)
```

```
mean(abs(res_df - AI_res_df) < 0.01)
```

```
## [1] 0.9111111
```

```
abs(res_df - AI_res_df) |> max()
```

```
## [1] 0.02967084
```

- The AI `bayesLogistic()` has the same high-level structure as my `bayesLogistic()`: find the posterior mode by Newton-Raphson, compute the Laplace log marginal likelihood, estimate posterior means by Metropolis-Hastings, fit a standard logistic regression with `glm()` for MLE, and return a list with the required fields.
- The AI output is correct on this data set. The result data show most of the case (93% of the values) has difference less than 0.01, and the max abs. diff. is 0.03.
- Similarly, my solution is better organized for a larger assignment because `bayesLogistic()` depends on reusable helper functions, and `main()` exports all needed helpers to the parallel workers. The AI version is understandable, but it places formulas directly in several functions, so changing the prior, likelihood, or numerical stabilization would require edits in multiple places.
- The AI Newton-Raphson helper `getMode()` has no maximum iteration limit. It converges here, but my `get_betas_mode_NR()` includes `max_iter`, tolerance, and an optional verbose message, which is safer if the data or starting values are less well behaved.
- Both `main()` functions use a list of per-predictor results returned from `clusterApply()`. Each result is a list containing the predictor index, log marginal likelihood, Bayesian beta estimates, and MLE beta estimates. The data structures and parallel algorithm are therefore very similar.